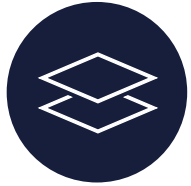




Entity Relations & Fetch Strategies

April 2026

Recap day 3



Day 3

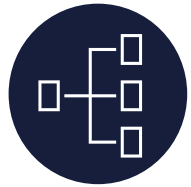
Abstraction layers, Hibernate entity mapping, CRUD, JPQL



Polymorphism in Hibernate: inheritance strategies, proxy pitfalls



Lazy loading: first introduction and `LazyInitializationException`



Today

UML, entity relationships, fetch strategies, and the Decorator pattern

Module overview



d

Day 1

JDBC & Object
Mapping



F

Day 2

Transactions &
Concurrency



Day 3

Abstraction Layers
& Hibernate



O

Day 4

Entity Relations &
Fetch Strategies



Day 5

ORM Caching &
Performance

Time planning



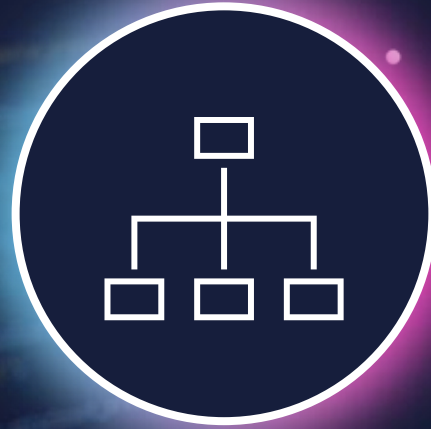
Starting time: 9.30



Breaks: ...



End time: 16.30



Section 1

UML for data modeling

Why UML?



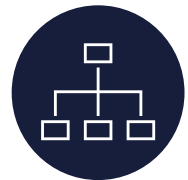
Real applications have entities with relationships



We need a way to think about and communicate these models



UML = Unified Modelling Language



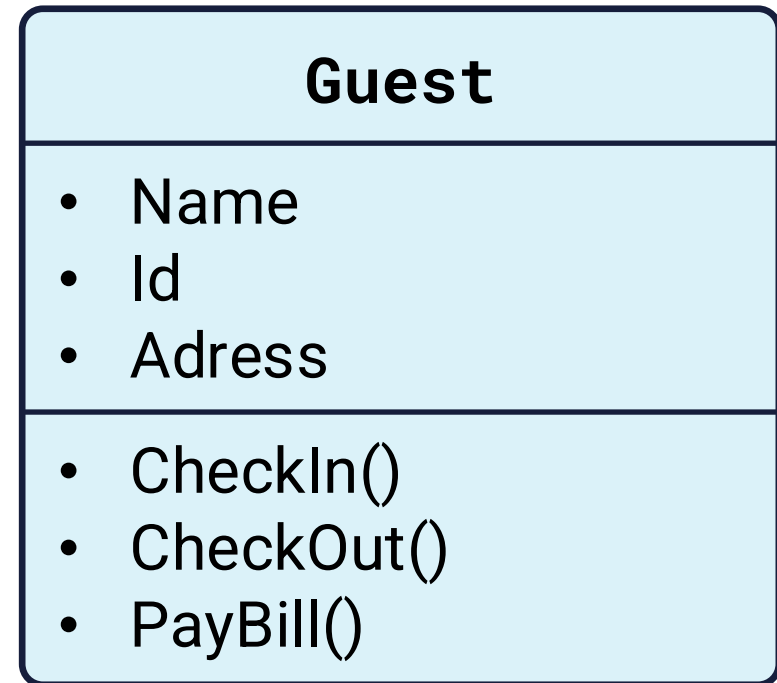
A standardized way to visualize system design



We only need the parts relevant for data modelling

Class diagrams: the essentials

- A box = one class or entity
- Top section: class name
- Middle section: attributes
- Bottom section: methods (less relevant for data modeling)



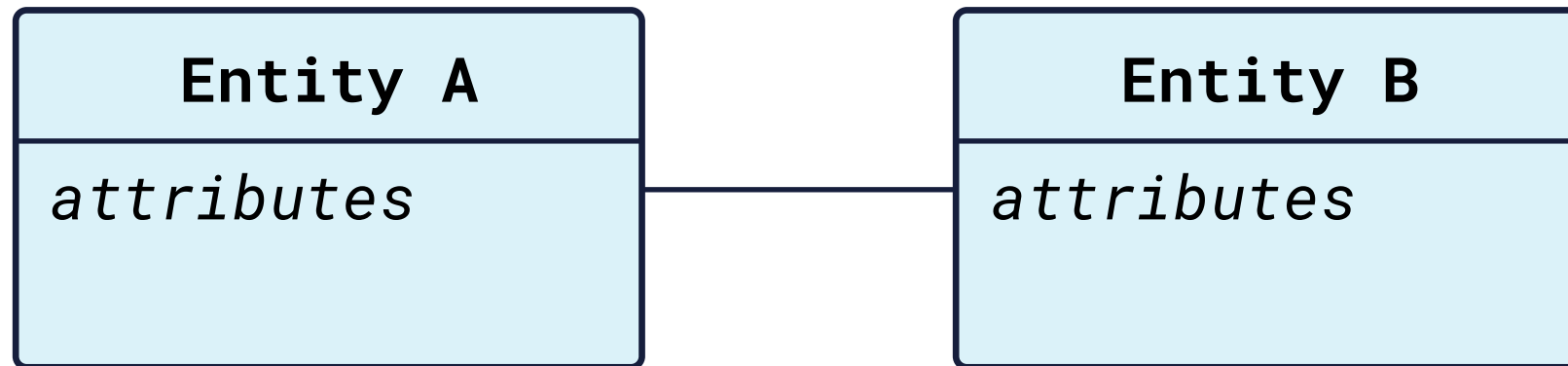
Attribute notation

- Format: `name: Type`
- Example: `name: String, age: int, email: String`
- Visibility markers (brief):
 - `+` public
 - `-` private
 - `#` protected
- For data modelling, visibility is secondary

Student
+ name: String - age: int # email: String

Relationships between entities

- Entities don't exist in isolation
- A line between two boxes = a relationship
- **Association**: a general connection between two entities
- The interesting part: what kind of relationship?





Relationship types

○→○

One-to-one

One student has one profile

b

One-to-many

One department has many courses

c

Many-to-one

Many enrollments belong to one student

k

Many-to-many

Students take many courses, courses have many students

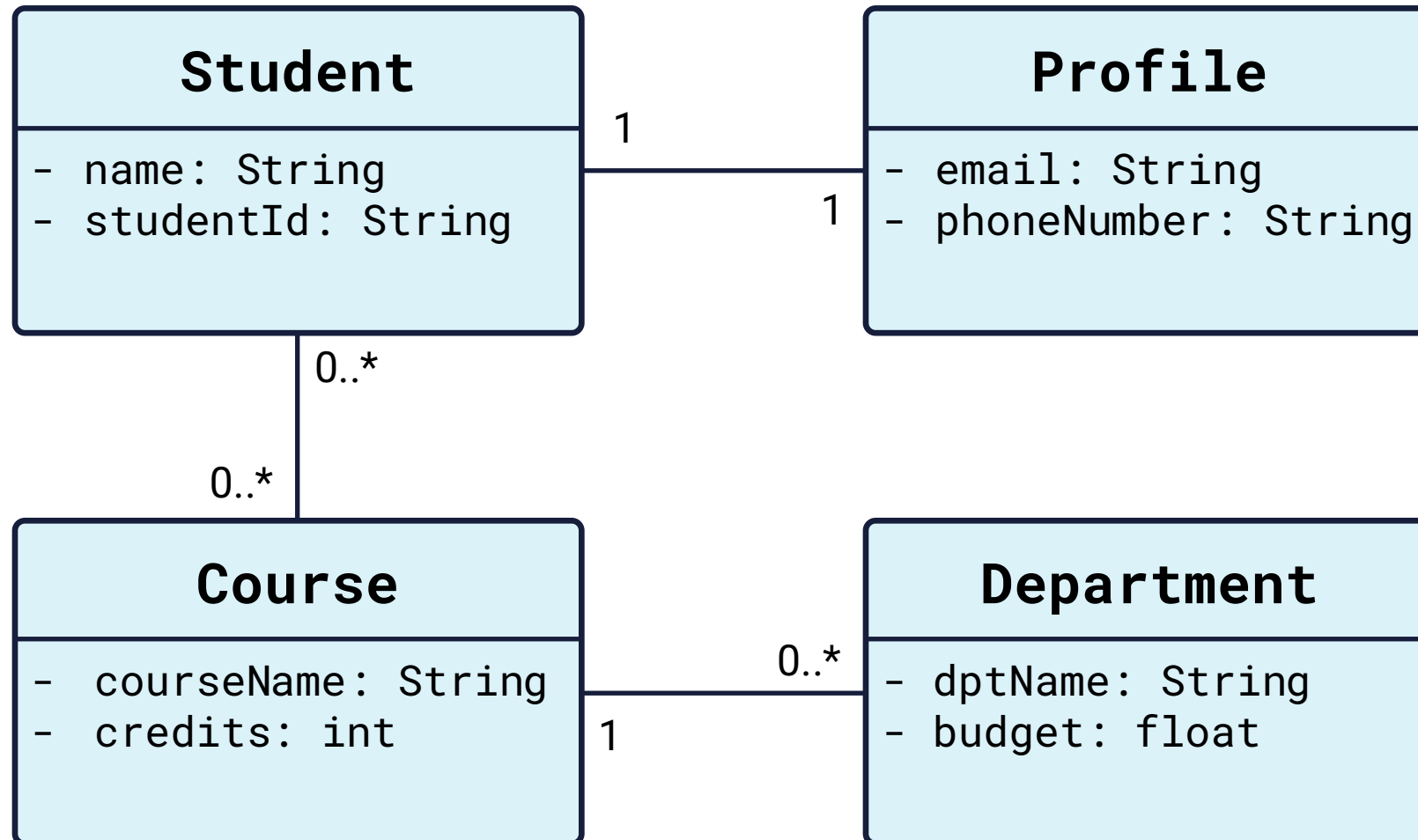
Multiplicity

Annotated at the ends of a relationship line

- **1** $\rightarrow$ exactly one
- **0..1** $\rightarrow$ zero or one (optional)
- ***** or **0..*** $\rightarrow$ zero or more
- **1..*** $\rightarrow$ one or more



Multiplicity



Navigability

Arrows show which side “knows about” the other

Unidirectional

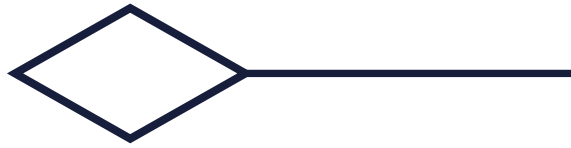
- One side has a reference
- Example: `Enrollment` → `Student`
(enrollment knows its student)

Bidirectional

- Both sides reference each other
- Example: `Student` ↔ `Enrollment`



Composition vs. aggregation

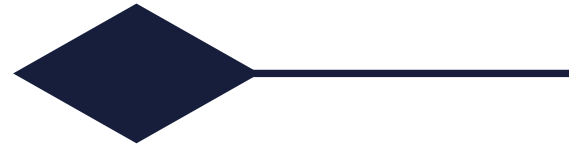


Aggregation

"has-a"

Child can exist independently

- A **Student** has **Courses**
- Deleting student doesn't delete courses



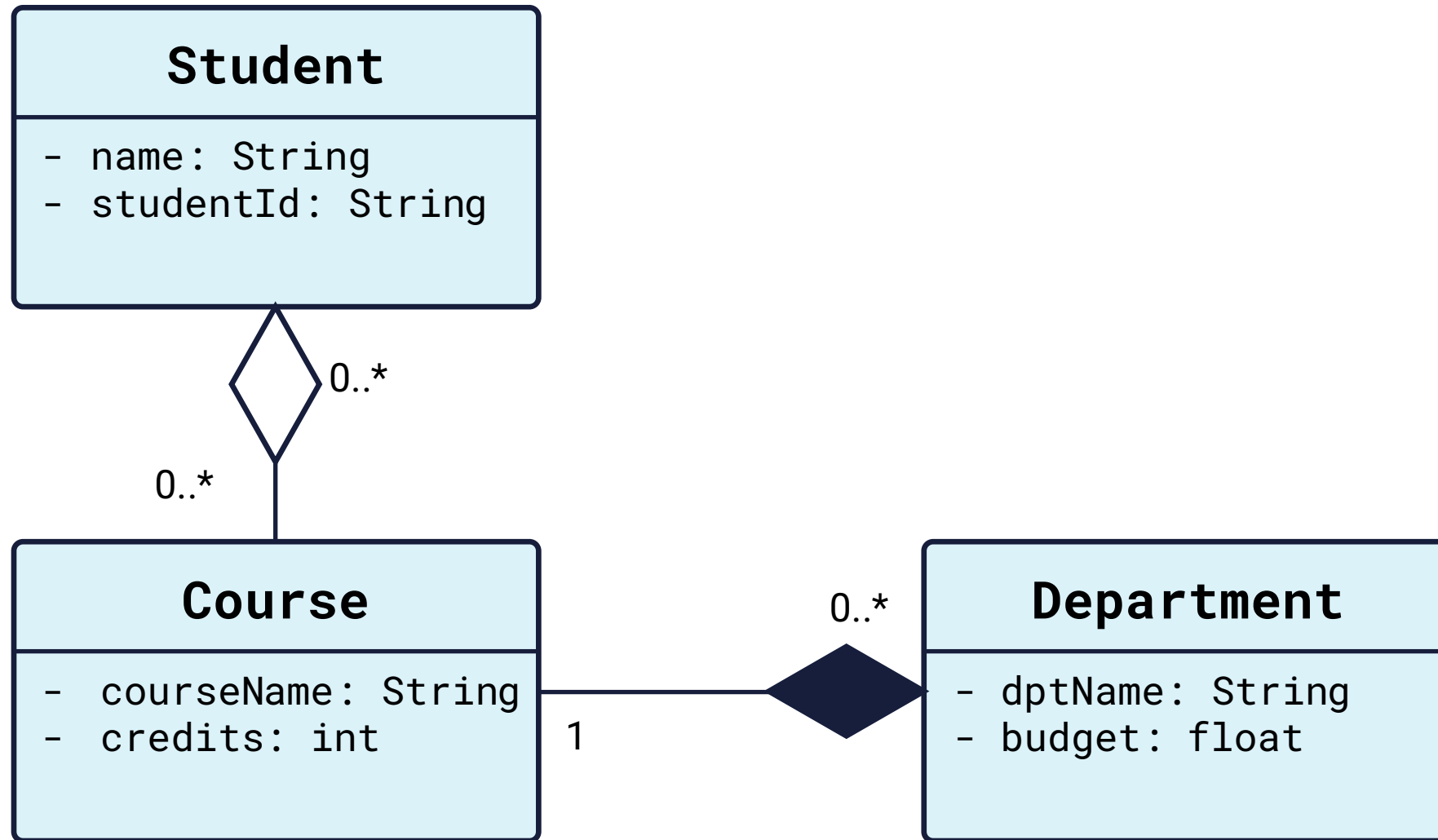
Composition

"owns"

Child cannot exist without parent

- A **Department** owns **Courses**
- Deleting department deletes courses

Diamond goes on parent/container side



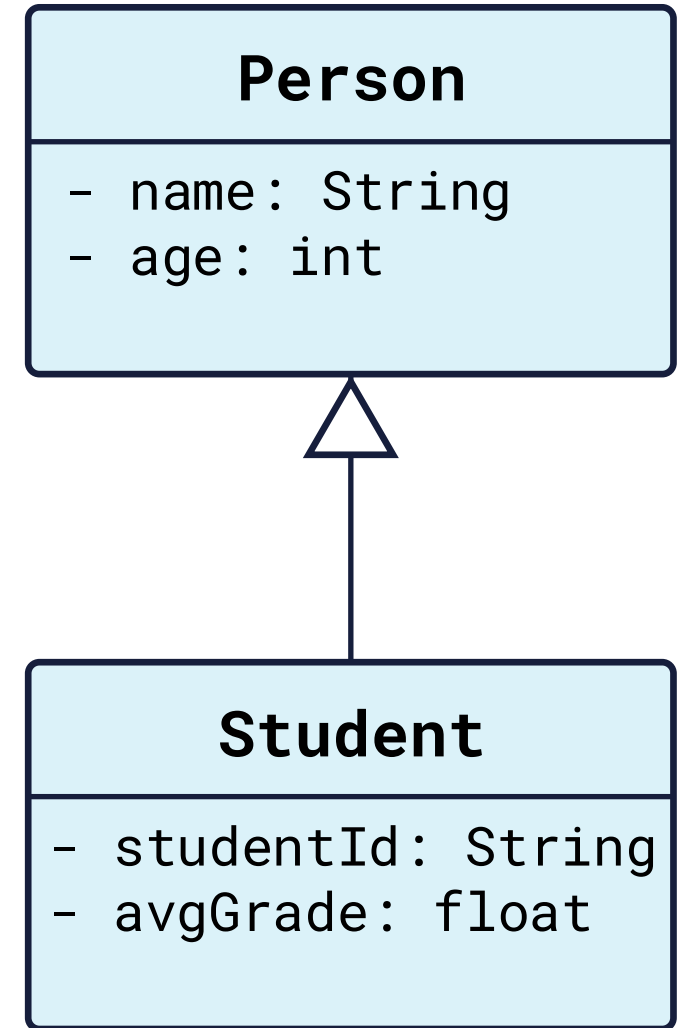
A note on composition vs. aggregation

- This distinction is much debated in UML
- Some practitioners argue aggregation adds little value over plain association
- In practice: focus on the lifecycle dependency
 - "If the parent is deleted, should the children be deleted too?"
- Don't overthink it! The important thing for us is to model the relationships clearly



Inheritance in UML

- Solid line with a hollow arrowhead pointing to the parent
- Maps to `extends` in Java
- We saw this on day 3: Hibernate inheritance strategies
- Single Table, Table Per Class, Table Per Concrete Class

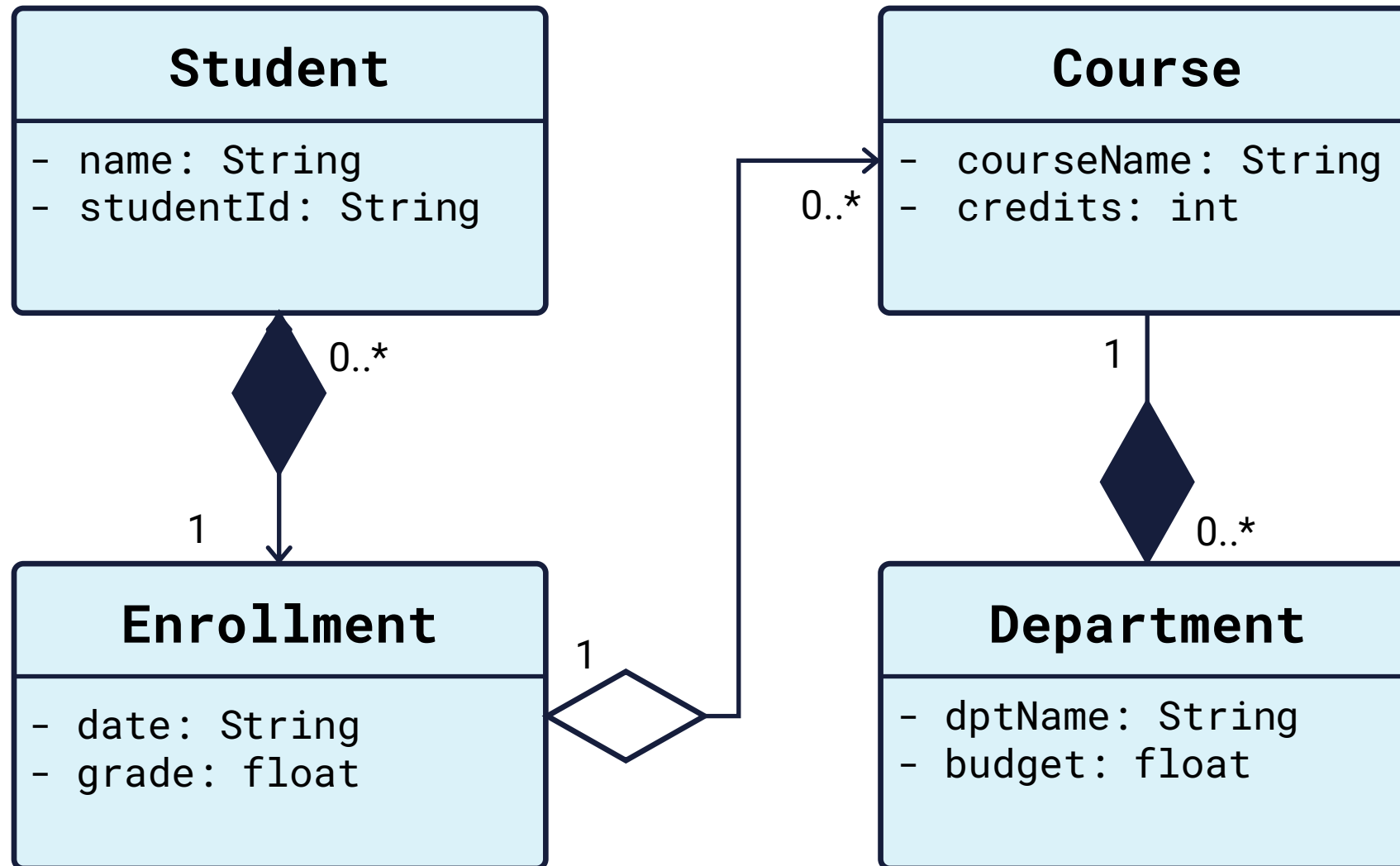


Reading a UML diagram

1. Identify the entities (boxes)
2. Identify the relationships (lines)
3. Read the multiplicity (numbers at each end)
4. Check navigability (arrows)
5. Translate to a mental model of database tables



Reading a UML diagram



Mini exercise

Drawing a UML diagram

- Given: a textual description of a university domain
- Draw a UML class diagram (paper or draw.io)
- Include entity names, key attributes, multiplicity
- Discuss: Composition vs aggregation? Undirectional vs bidirectional?
- Compare diagrams with a partner



Season 2

LEGEND
☆☆☆
SINCE 1971

From UML to code



The UML diagram tells us the “what”



Now we implement the “how”



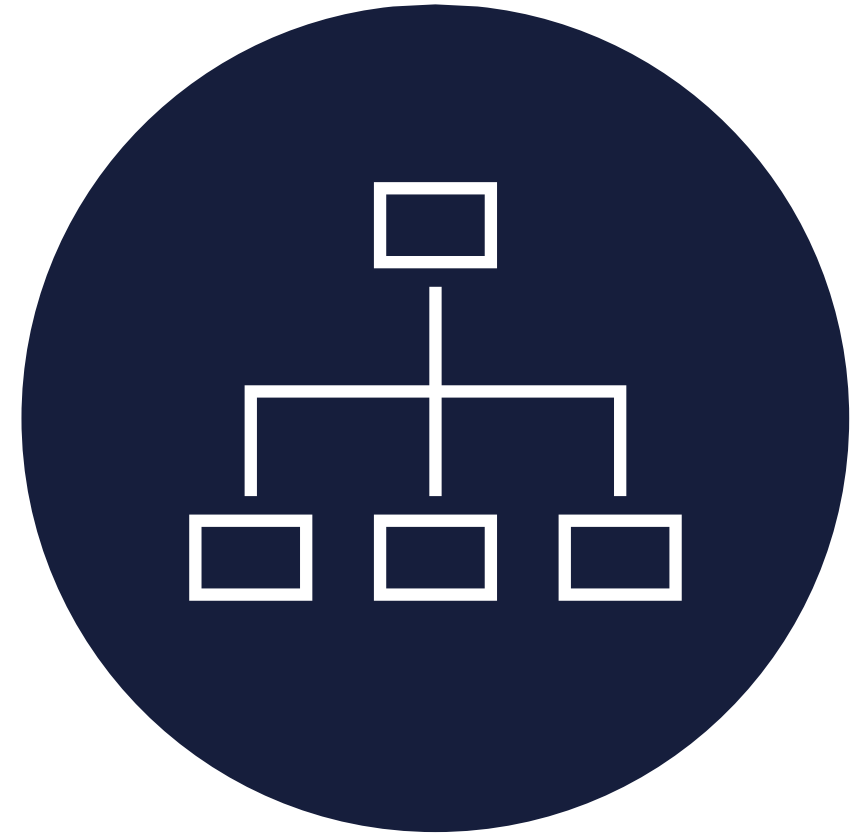
Hibernate provides annotations for every relationship type



The key question: who owns the relationship?

@ManyToOne: the child side

- Placed on the entity that holds the foreign key
- Example: `Enrollment` has a FK to `Student`
- `@ManyToOne` on `Enrollment.student`
- This is the **owning side** of the relationship
- `@JoinColumn(name = "student_id")` specifies the FK column



@OneToMany: the parent side

- Placed on the entity with the collection
- Example: `Student` has a `List<Enrollment>`
- `@OneToMany(mappedBy = "student")`
- `mappedBy` = "I don't own this relationship, the other side does"
- The `mappedBy` value is the field name on the owning side



Common mistake: forgetting mappedBy



Without `mappedBy`, Hibernate thinks it's two separate relationships



It creates an unwanted join table (e.g., `student_enrollments`)



Instead of using the FK on the child table



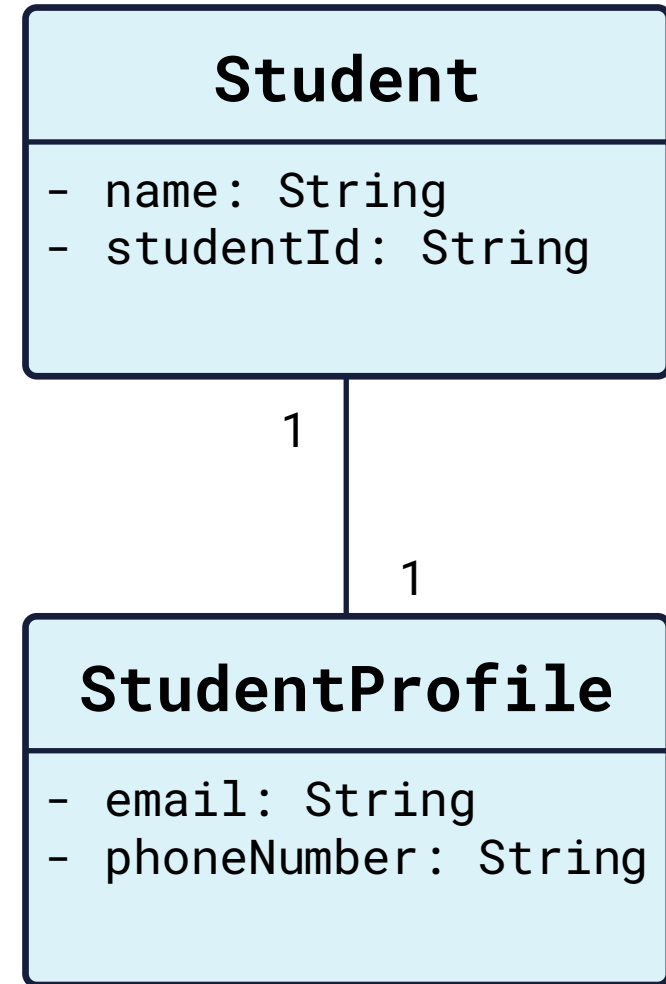
Symptoms: unexpected tables, duplicate data, confusion



Rule: always specify `mappedBy` on the non-owning side

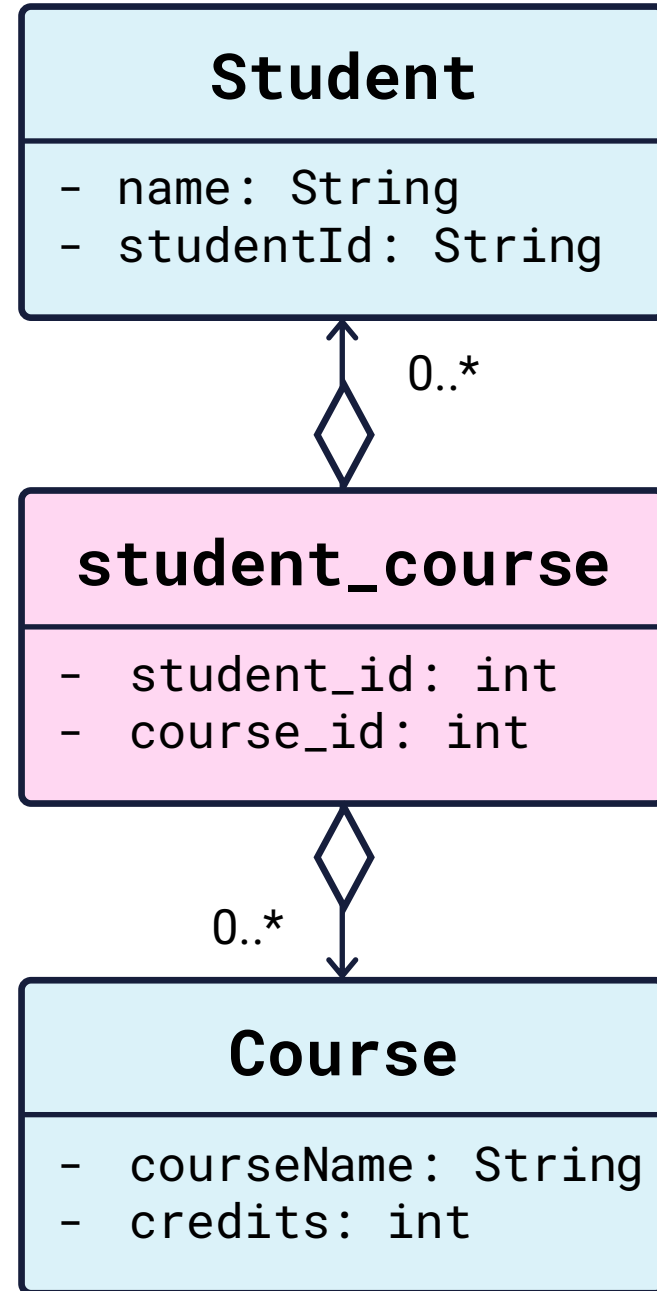
@OneToOne

- Similar to @ManyToOne but with a uniqueness constraint
- Example: `Student` has one `StudentProfile`
- Can be unidirectional or bidirectional
- Same `mappedBy` logic applies
- The side with the FK is the owning side



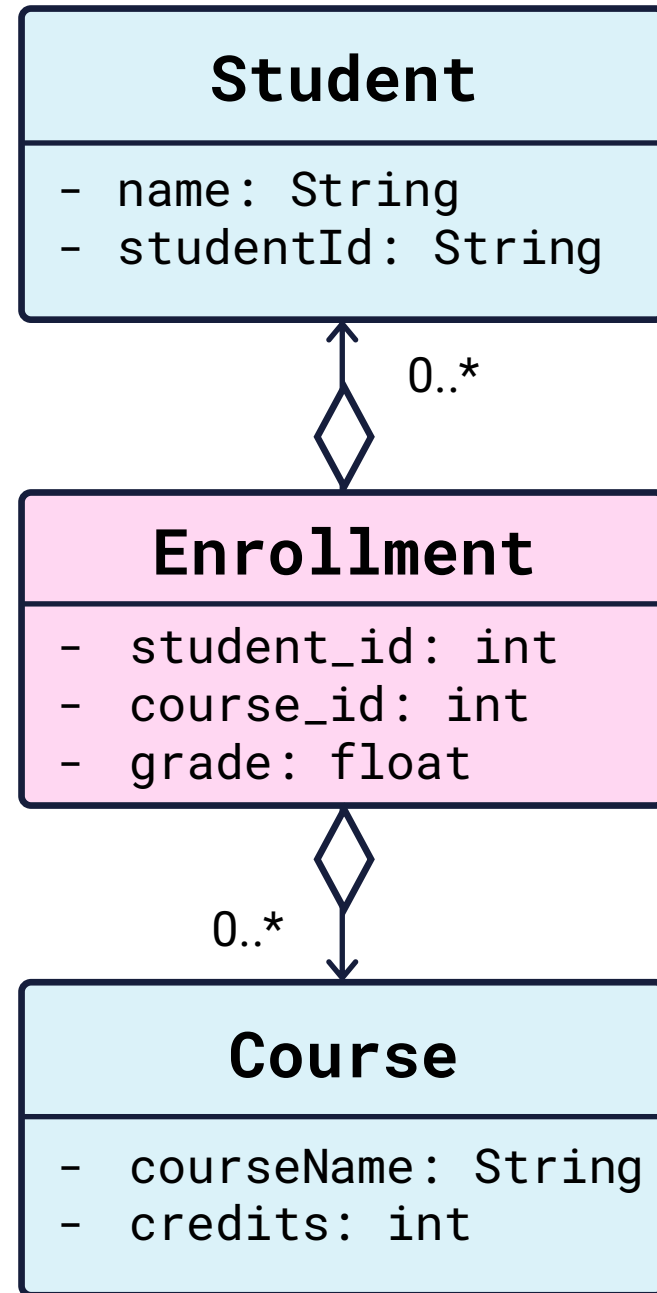
@ManyToMany

- Example: students and courses (without extra attributes)
- Requires a **join table** (e.g., `student_course`)
- `@JoinTable` configures the join table and its columns
- One side is the owner (has `@JoinTable`)
- The other side has `mappedBy`



@ManyToMany: the practical alternative

- In practice: prefer modelling as two @ManyToOne with an explicit join entity
- Example: **Enrollment** entity with FK to **Student** and FK to **Course**
- Advantages:
 - Extra attributes on the relationship (e.g., enrollment date, grade)
 - More control over the lifecycle
 - Easier to query

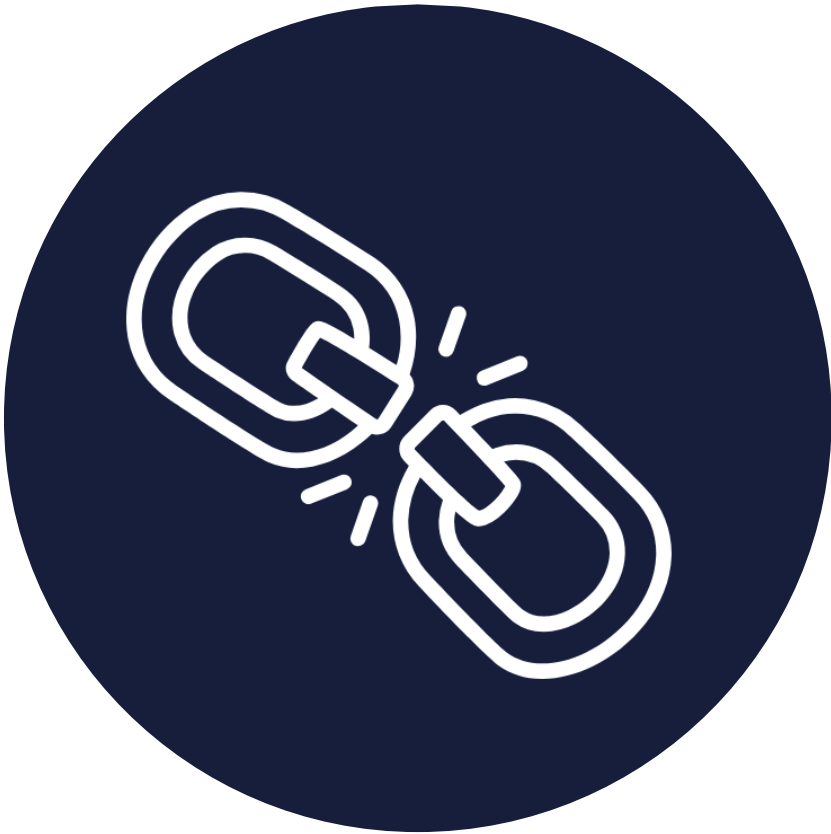


Cascade types

What happens to children when you persist, merge, or remove a parent?

- `CascadeType.PERSIST`
→ saving parent saves children
- `CascadeType.MERGE`
→ updating parent updates children
- `CascadeType.REMOVE`
→ deleting parent deletes children
- `CascadeType.ALL`
→ all of the above (use with care!)

orphanRemoval



- `orphanRemoval = true` on `@OneToMany`
- If a child is removed from the collection, delete it from the database
- Different from `CascadeType.REMOVE`:
 - `REMOVE`: parent deleted → children deleted
 - `orphanRemoval`: child removed from collection → child deleted
- Golden rule: only cascade parent → child, never child → parent

Bidirectional relationship maintenance

- Both sides must be kept in sync in Java code
- Adding an enrollment to a student's list:
 - Also set `enrollment.setStudent(student)`
- Hibernate only checks the owning side for persistence
- But your in-memory object graph should be consistent



```
// In Student class:  
public void addEnrollment(Enrollment enrollment) {  
    enrollments.add(enrollment);  
    enrollment.setStudent(this);  
}
```



Helper methods for bidirectional sync

- Convenience method keeps both sides in sync
- Keeps the synchronization logic in one place
- Call `student.addEnrollment(e)` instead of doing it manually

Demo

Entity relationships in Hibernate

- `Student` with `@OneToMany` to `Enrollment`
- `Enrollment` with `@ManyToOne` to `Student` and `Course`
- Persist a student with enrollments using cascade
- View the generated SQL (joins, FKs)
- See what happens without `mappedBy`



Mini exercise: mapping entity relationships

- Create entities and relations from your UML diagram (previous exercise)
- Add cascade on persist where applicable
- Create the different objects, set the relations and, persist via cascade
- Verify the data in the database
- **Bonus:** add a helper method



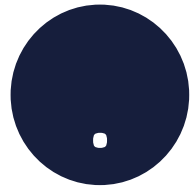
Sep 3

LEGEND
★★★
SINCE 1971

Recap: lazy loading from day 3



Day 3: first introduction to lazy loading



Collections are not loaded until you access them



If the Session is closed when you access them → `LazyInitializationException`



Today: going much deeper into fetch strategies

The N+1 problem

- The most common ORM performance problem
- You load 10 students (lazy), and access the enrollments of each Hibernate fires: 1 query for students + 10 queries for enrollments
- = 11 queries for what should be 1 or 2
- 100 students → 101 queries
- 1000 students → 1001 queries



Eager vs lazy fetching



FetchType.LAZY

- load data only when accessed
 - Fast initial load
- Risk: N+1 problem, LazyInitializationException



FetchType.EAGER

- Load data immediately with the parent
 - No N+1 for this relationship
- Risk: loading too much data, always

Default fetch types

Annotation	Default
@OneToMany	LAZY
@ManyToMany	LAZY
@ManyToOne	EAGER
@OneToOne	EAGER

Proxies revisited: single-valued associations



For `@ManyToOne` / `@OneToOne` (when lazy): Hibernate creates a proxy



The proxy is a subclass of your entity, generated at runtime



Contains only the ID initially



Accessing any other property triggers a SELECT

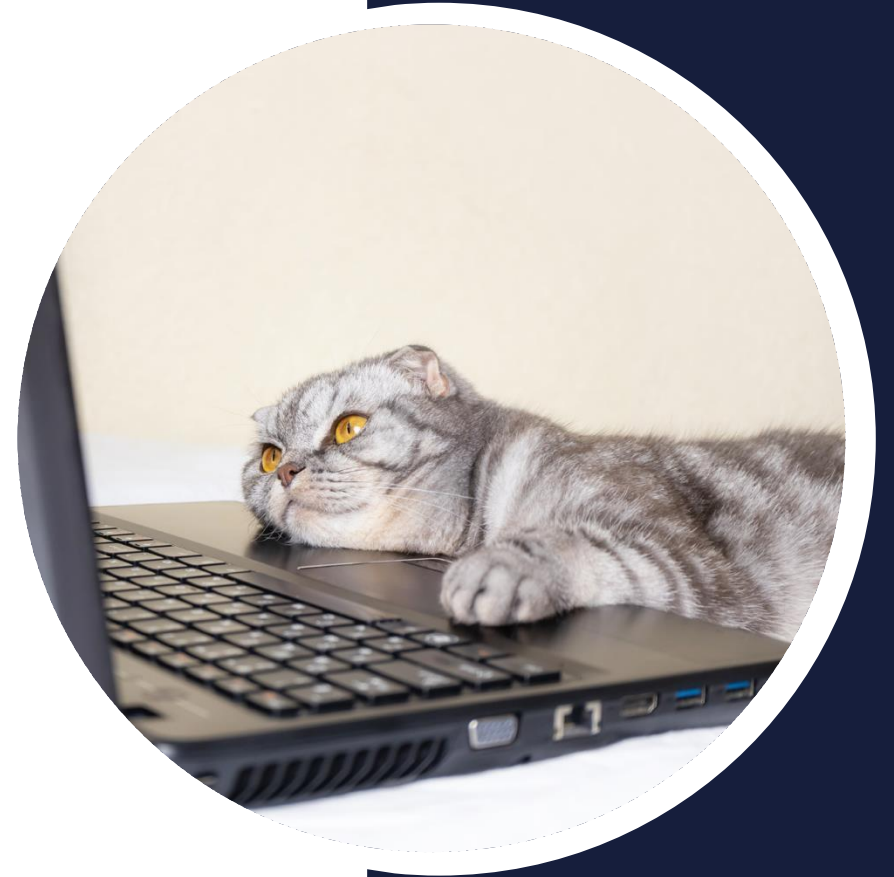


Same proxy issues as day 3 (`instanceof`, casting)

PersistentCollections

How collections are lazified

- For `@OneToMany` / `@ManyToMany`:
Hibernate wraps your collection
- Your `List<Enrollment>` becomes a
Hibernate `PersistentList`
- Looks like a normal list, but fires a
SELECT on first access
- Triggered by: `.size()`, `.get()`, iteration,
`.isEmpty()`



LazyInitializationException explained



Proxies and PersistentCollections need an active Session



Session closed + access unloaded data = exception



Hibernate says: "I can't fetch this, I have no database connection"



Common scenario: loading data in a service, using it in a controller after Session is closed

```
SELECT s FROM Student s JOIN FETCH s.enrollments
```



Solving N+1: JOIN FETCH

- Loads students AND enrollments in a single SQL JOIN
- Efficient: one query instead of N+1
- Limitation: only one collection at a time
 - Multiple JOIN FETCHes → Cartesian product

Solving N+1: @BatchSize

- `@BatchSize(size = 10)` on the collection field
- Instead of loading enrollments per student, load in batches
- 10 students with batch 10 = 2 queries total
 - 1 for students
 - 1 for enrollments: `WHERE student_id IN (?, ?, ...)`
- A general safety net against N+1





Entity Graphs (brief)



A JPA API to define fetch plans dynamically



Specify which relationships to load eagerly per query



Override the default fetch strategy at query time



Useful for complex use cases, but beyond today's scope

Which strategy when?

- **JOIN FETCH:** you know you need the related data
 - Targeted, efficient, per-query
- **@BatchSize:** general safety net across the application
 - Set it and (mostly) forget it
- They're not mutually exclusive → use both
- Default recommendation: LAZY everywhere + JOIN FETCH/BatchSize as needed



The decorator pattern and proxies

How do Hibernate proxies work under the hood?

- They use a structure similar to the **Decorator pattern**
- Formally: this is the **Proxy pattern** — same mechanics, different intent
- Decorator: add behavior without modifying the original class
- Proxy: control access to the original object



How proxies work: the pattern



An interface or base class (e.g., `Student`)



A proxy/decorator that extends or implements the same type



Intercepts method calls



Adds behavior (lazy loading) before/after delegating



Hibernate uses bytecode generation (ByteBuddy) to create these at runtime

Demo

Fetch strategies and the N+1 problem

- Load students with lazy enrollments
- Enable `hibernate.show_sql=true` and count queries
- Demonstrate N+1: iterate enrollments → many queries
- Fix with JOIN FETCH → single query
- Fix with `@BatchSize` → batched queries
- Inspect a proxy in the debugger



Mini exercise: fetch strategies in practice

- Load all students, iterate their enrollments → count the SQL queries (N+1!)
- Write a `JOIN FETCH` JPQL query → verify query count drops
- Add `@BatchSize(size = 5)` to enrollments → observe batched queries
- Change a `@ManyToOne` from EAGER to LAZY → observe the difference
- **Bonus:** close the Session, access a lazy collection → handle the `LazyInitializationException`

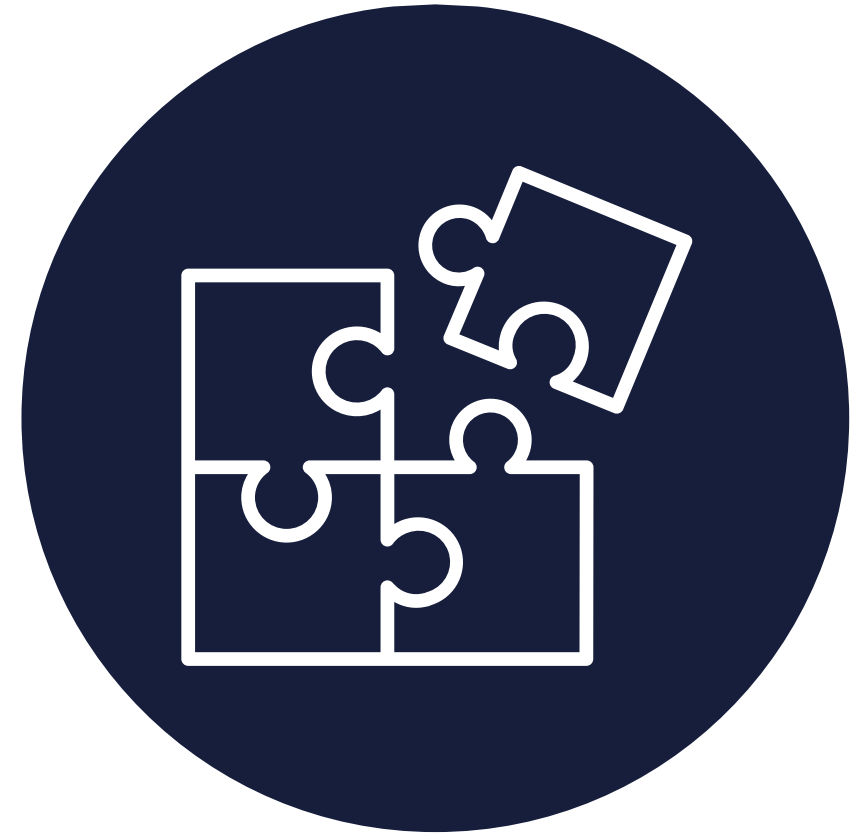


Section 4

The decorator pattern — deep dive

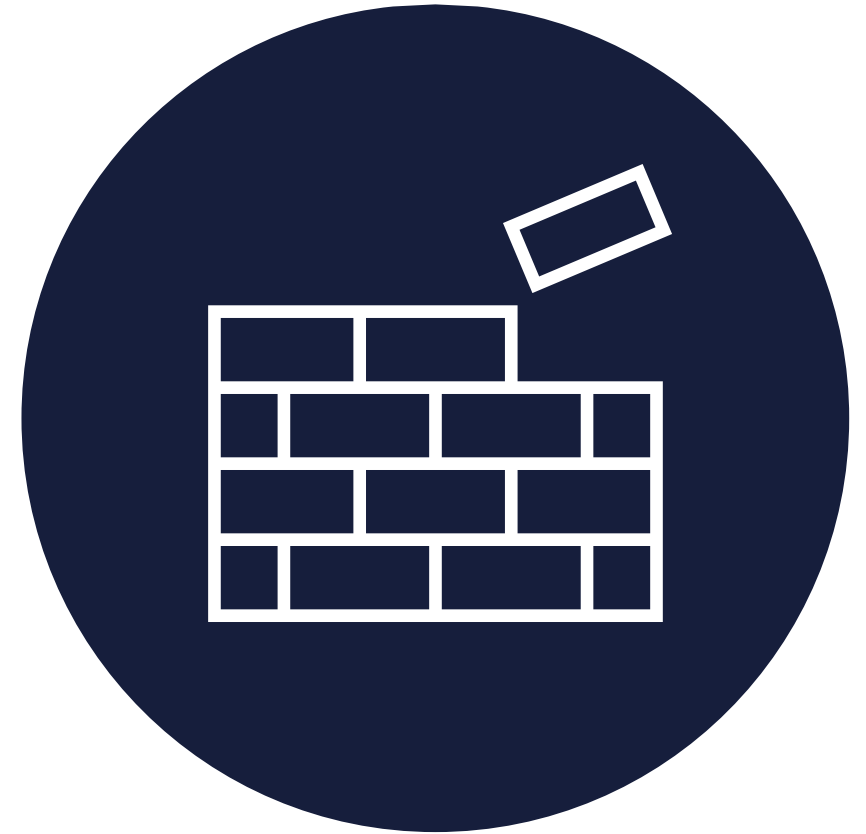
From proxies to a general pattern

- We saw the Decorator/Proxy pattern in the context of Hibernate
- The Decorator pattern is useful far beyond ORMs
- A general-purpose pattern for adding behavior without modifying existing code
- Let's look at it as something you can use in your own code



The problem Decorator solves

- You want to add behavior without changing the original class
- Inheritance works, but it's static
 - Can't mix and match at runtime
- Example: a **Repository** that needs logging, caching, and timing
 - Without decorator: modify the Repository class directly or create many subclasses



Decorator pattern structure

- **Component:** the interface (e.g., `Repository<T>`)
- **Concrete Component:** the real implementation (e.g., `HibernateRepository<T>`)
- **Decorator:** wraps a Component, implements the same interface
- **Concrete Decorators:** each adds one behavior
 - `LoggingRepository<T>`
 - `CachingRepository<T>`
 - `TimingRepository<T>`

How it works: code example



```
public class LoggingRepository<T> implements Repository<T> {  
    private final Repository<T> delegate;  
  
    public LoggingRepository(Repository<T> delegate) {  
        this.delegate = delegate;  
    }  
  
    @Override  
    public T findById(long id) {  
        System.out.println("Finding entity by id: " + id);  
        T result = delegate.findById(id);  
        System.out.println("Found: " + result);  
        return result;  
    }  
}
```

```
Repository<Student> repo =  
    new LoggingRepository<>(  
        new TimingRepository<>(  
            new CachingRepository<>(  
                new HibernateRepository<>()))));
```



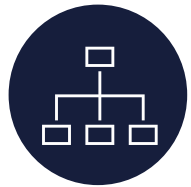
Composing decorators

- Each decorator adds one responsibility
- Order matters!
- Follows the Single Responsibility Principle

A note on Hibernate proxies



Hibernate proxies don't use composition like our decorator example



They extend the entity class and intercept via bytecode generation



Same conceptual pattern, different implementation mechanism



Both wrap/intercept behavior around an underlying object

Decorator vs inheritance

- **Inheritance:** static, determined at compile time
- **Decorator:** dynamic, composable at runtime
- Inheritance leads to class explosion:
 - `LoggingCachingRepository`
 - `LoggingTimingRepository`
 - `CachingTimingRepository`
 - `LoggingCachingTimingRepository...`
- With decorators: just compose at runtime



Decorator pattern in the wild



Java I/O

```
BufferedReader(InputStreamReader(FileInputStream(...)))
```



Java Collections

```
Collections.unmodifiableList(), Collections.synchronizedList()
```



Spring Framework

AOP proxies, transaction management, security



Your own framework

Add cross-cutting concerns without modifying core code

Connection to Hibernate proxies

- Hibernate proxies add "load on access" behavior
- They wrap/extend your entity and intercept method calls
- Understanding decorators makes proxy behavior predictable:
 - `instanceof` issues: the proxy is a wrapping subclass
 - Lazy loading: the intercepted call triggers data loading
 - The casting issues from day 3 now make complete sense

Demo

Build a decorator

- Given: `Repository<T>` interface (`findById`, `findAll`, `save`, `delete`)
- And: `SimpleRepository<T>` concrete implementation
- Create a `LoggingRepository<T>` → log method name + arguments before delegating
- Create a `TimingRepository<T>` → measure and print execution time
- Compose: `new LoggingRepository(new TimingRepository(new SimpleRepository()))`
- Verify both behaviors work together
- **Bonus:** create a `CachingRepository<T>` with `findById` cached in a `Map`

What we covered today

- UML class diagrams for data modeling
- Entity relationships in Hibernate:
 - `@OneToMany`, `@ManyToOne`, `@OneToOne`, `@ManyToMany`
 - Cascading, `orphanRemoval`, bidirectional sync
- Fetch strategies:
 - Lazy vs eager, N+1, JOIN FETCH, `@BatchSize`
- Proxies explained through the Decorator pattern
- The Decorator pattern as a general design pattern



This afternoon: your framework project

O

Support entity relationships in your own framework

- Define and persist relationships between entities



Implement a fetch strategy

- Choose eager or lazy, and build the mechanism

P

Apply what you learned this morning to your own code